- If the level isn't complete, then the method sets the game state to **GameState::Active**, and returns.

Update the game world

In this sample, when the game is running, the **Simple3DGame::UpdateDynamics** method is called from the **Simple3DGame::RunGame** method (which is called from **GameMain::Update**) to update objects that are rendered in a game scene.

A loop such as **UpdateDynamics** calls any methods that are used to set the game world in motion, independent of the player input, to create an immersive game experience and make the level come alive. This includes graphics that needs to be rendered, and running animation loops to bring about a dynamic world even when there's no player input. In your game, that could include trees swaying in the wind, waves cresting along shore lines, machinery smoking, and alien monsters stretching and moving around. It also encompasses the interaction between objects, including collisions between the player sphere and the world, or between the ammo and the obstacles and targets.

Except when the game is specifically paused, the game loop should continue updating the game world; whether that's based on game logic, physical algorithms, or whether it's just plain random.

In the sample game, this principle is called *dynamics*, and it encompasses the rise and fall of the pillar obstacles, and the motion and physical behaviors of the ammo spheres as they are fired and in motion.

The Simple3DGame::UpdateDynamics method

This method deals with these four sets of computations.

- The positions of the fired ammo spheres in the world.
- The animation of the pillar obstacles.
- The intersection of the player and the world boundaries.
- The collisions of the ammo spheres with the obstacles, the targets, other ammo spheres, and the world.

The animation of the obstacles takes place in a loop defined in the **Animate.h/.cpp** source code files. The behavior of the ammo and any collisions are defined by simplified physics algorithms, supplied in the code and parameterized by a set of global constants for the game world, including gravity and material properties. This is all computed in the game world coordinate space.

Review the flow

Now that we've updated all of the objects in the scene, and calculated any collisions, we need to use this info to draw the corresponding visual changes.

After **GameMain::Update** has completed the current iteration of the game loop, the sample immediately calls **GameRenderer::Render** to take the updated object data and generate a new scene to present to the player, as shown below.

C++/WinRT

```
void GameMain::Run()
{
    while (!m_windowClosed)
    {
        if (m_visible)
        {
            switch (m_updateState)
            {
            case UpdateEngineState::Deactivated:
            case UpdateEngineState::TooSmall:
                ...
                // Otherwise, fall through and do normal processing to
perform rendering.
            default:

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(
                    CoreProcessEventsOption::ProcessAllIfPresent);
                // GameMain::Update calls Simple3DGame::RunGame. If game is
in Dynamics
                // state, uses Simple3DGame::UpdateDynamics to update game
world.
                Update();
                // Render is called immediately after the Update loop.
                m_renderer->Render();
                m_deviceResources->Present();
                m_renderNeeded = false;
            }
        }
        else
        {
            CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(
                CoreProcessEventsOption::ProcessOneAndAllPending);
        }
    }
    m_game->OnSuspending();  // Exiting due to window close, so save state.
}
```

Render the game world's graphics

We recommend that the graphics in a game update often, ideally exactly as often as the main game loop iterates. As the loop iterates, the game world's state is updated, with or without player input. This allows the calculated animations and behaviors to be displayed smoothly. Imagine if we had a simple scene of water that moved only when the player pressed a button. That wouldn't be realistic; a good game looks smooth and fluid all the time.

Recall the sample game's loop as shown above in **GameMain::Run**. If the game's main window is visible, and isn't snapped or deactivated, then the game continues to update and render the results of that update. The **GameRenderer::Render** method we examine next renders a representation of that state. This is done immediately after a call to **GameMain::Update**, which includes **Simple3DGame::RunGame** to update states, as discussed in the previous section.

GameRenderer::Render draws the projection of the 3D world, and then draws the Direct2D overlay on top of it. When completed, it presents the final swap chain with the combined buffers for display.

> ⓘ **Note**
>
> There are two states for the sample game's Direct2D overlay—one where the game displays the game info overlay that contains the bitmap for the pause menu, and one where the game displays the crosshairs along with the rectangles for the touchscreen move-look controller. The score text is drawn in both states. For more information, see **Rendering framework I: Intro to rendering**.

The GameRenderer::Render method

C++/WinRT

```
void GameRenderer::Render()
{
    bool stereoEnabled{ m_deviceResources->GetStereoState() };

    auto d3dContext{ m_deviceResources->GetD3DDeviceContext() };
    auto d2dContext{ m_deviceResources->GetD2DDeviceContext() };

    ...
        if (m_game != nullptr && m_gameResourcesLoaded &&
m_levelResourcesLoaded)
        {
            // This section is only used after the game state has been
initialized and all device
            // resources needed for the game have been created and
associated with the game objects.
```

```
            ...
            for (auto&& object : m_game->RenderObjects())
            {
                object->Render(d3dContext,
m_constantBufferChangesEveryPrim.get());
            }
        }

        d3dContext->BeginEventInt(L"D2D BeginDraw", 1);
        d2dContext->BeginDraw();

        // To handle the swapchain being pre-rotated, set the D2D
transformation to include it.
        d2dContext->SetTransform(m_deviceResources-
>GetOrientationTransform2D());

        if (m_game != nullptr && m_gameResourcesLoaded)
        {
            // This is only used after the game state has been initialized.
            m_gameHud.Render(m_game);
        }

        if (m_gameInfoOverlay.Visible())
        {
            d2dContext->DrawBitmap(
                m_gameInfoOverlay.Bitmap(),
                m_gameInfoOverlayRect
                );
        }
        ...
    }
}
```

The Simple3DGame class

These are the methods and data members that are defined by the **Simple3DGame** class.

Member functions

Public member functions defined by **Simple3DGame** include the ones below.

- **Initialize**. Sets the starting values of the global variables, and initializes the game objects. This is covered in the Initialize and start the game section.
- **LoadGame**. Initializes a new level, and starts loading it.
- **LoadLevelAsync**. A coroutine that initializes the level, and then invokes another coroutine on the renderer to load the device-specific level resources. This method runs in a separate thread; as a result, only **ID3D11Device** methods (as opposed to **ID3D11DeviceContext** methods) can be called from this thread. Any device context

methods are called in the **FinalizeLoadLevel** method. If you're new to asynchronous programming, then see Concurrency and asynchronous operations with C++/WinRT.

- **FinalizeLoadLevel**. Completes any work for level loading that needs to be done on the main thread. This includes any calls to Direct3D 11 device context (**ID3D11DeviceContext**) methods.
- **StartLevel**. Starts the gameplay for a new level.
- **PauseGame**. Pauses the game.
- **RunGame**. Runs an iteration of the game loop. It's called from **App::Update** one time every iteration of the game loop if the game state is **Active**.
- **OnSuspending** and **OnResuming**. Suspend/resume the game's audio, respectively.

Here are the private member functions.

- **LoadSavedState** and **SaveState**. Load/save the current state of the game, respectively.
- **LoadHighScore** and **SaveHighScore**. Load/save the high score across games, respectively.
- **InitializeAmmo**. Resets the state of each sphere object used as ammunition back to its original state for the beginning of each round.
- **UpdateDynamics**. This is an important method because it updates all the game objects based on canned animation routines, physics, and control input. This is the heart of the interactivity that defines the game. This is covered in the Update the game world section.

The other public methods are property accessor that return gameplay- and overlay-specific information to the app framework for display.

Data members

These objects are updated as the game loop runs.

- **MoveLookController** object. Represents the player input. For more information, see Adding controls.
- **GameRenderer** object. Represents a Direct3D 11 renderer, which handles all the device-specific objects and their rendering. For more information, see Rendering framework I.
- **Audio** object. Controls the audio playback for the game. For more information, see Adding sound.

The rest of the game variables contain the lists of the primitives, and their respective in-game amounts, and game play specific data and constraints.

Next steps

We have yet to talk about the actual rendering engine—how calls to the **Render** methods on the updated primitives get turned into pixels on your screen. Those aspects are covered in two parts—Rendering framework I: Intro to rendering and Rendering framework II: Game rendering. If you're more interested in how the player controls update the game state, then see Adding controls.

Rendering framework I: Intro to rendering

Article • 10/20/2022

> **Note**
>
> This topic is part of the **Create a simple Universal Windows Platform (UWP) game with DirectX** tutorial series. The topic at that link sets the context for the series.

So far we've covered how to structure a Universal Windows Platform (UWP) game, and how to define a state machine to handle the flow of the game. Now it's time to learn how to develop the rendering framework. Let's look at how the sample game renders the game scene using Direct3D 11.

Direct3D 11 contains a set of APIs that provide access to the advanced features of high-performance graphic hardware that can be used to create 3D graphics for graphics-intensive applications such as games.

Rendering game graphics on-screen means basically rendering a sequence of frames on-screen. In each frame, you have to render objects that are visible in the scene, based on the view.

In order to render a frame, you have to pass the required scene information to the hardware so that it can be displayed on the screen. If you want to have anything displayed on screen, you need to start rendering as soon as the game starts running.

Objectives

To set up a basic rendering framework to display the graphics output for a UWP DirectX game. You can loosely break that down into these three steps.

1. Establish a connection to the graphics interface.
2. Create the resources needed to draw the graphics.
3. Display the graphics by rendering the frame.

This topic explains how graphics are rendered, covering steps 1 and 3.

Rendering framework II: Game rendering covers step 2—how to set up the rendering framework, and how data is prepared before rendering can happen.

Get started

It's a good idea to familiarize yourself with basic graphics and rendering concepts. If you're new to Direct3D and rendering, see Terms and concepts for a brief description of the graphics and rendering terms used in this topic.

For this game, the **GameRenderer** class represents the renderer for this sample game. It's responsible for creating and maintaining all the Direct3D 11 and Direct2D objects used to generate the game visuals. It also maintains a reference to the **Simple3DGame** object used to retrieve the list of objects to render, as well as status of the game for the heads-up display (HUD).

In this part of the tutorial, we'll focus on rendering 3D objects in the game.

Establish a connection to the graphics interface

For info about accessing the hardware for rendering, see the Define the game's UWP app framework topic.

The App::Initialize method

The **std::make_shared** function, as shown below, is used to create a **shared_ptr** to **DX::DeviceResources**, which also provides access to the device.

In Direct3D 11, a device is used to allocate and destroy objects, render primitives, and communicate with the graphics card through the graphics driver.

C++/WinRT

```
void Initialize(CoreApplicationView const& applicationView)
{
    ...

    // At this point we have access to the device.
    // We can create the device-dependent resources.
    m_deviceResources = std::make_shared<DX::DeviceResources>();
}
```

Display the graphics by rendering the frame

The game scene needs to render when the game is launched. The instructions for rendering start in the **GameMain::Run** method, as shown below.

The simple flow is this.

1. **Update**
2. **Render**
3. **Present**

GameMain::Run method

C++/WinRT

```
void GameMain::Run()
{
    while (!m_windowClosed)
    {
        if (m_visible) // if the window is visible
        {
            switch (m_updateState)
            {
            ...
            default:

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessAllIfPresent);
                Update();
                m_renderer->Render();
                m_deviceResources->Present();
                m_renderNeeded = false;
            }
        }
        else
        {

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessOneAndAllPending);
        }
    }
    m_game->OnSuspending();  // Exiting due to window close, so save state.
}
```

Update

See the Game flow management topic for more information about how game states are updated in the **GameMain::Update** method.

Render

Rendering is implemented by calling the **GameRenderer::Render** method from **GameMain::Run**.

If stereo rendering is enabled, then there are two rendering passes—one for the left eye and one for the right. In each rendering pass, we bind the render target and the depth-stencil view to the device. We also clear the depth-stencil view afterward.

> **Note**
>
> Stereo rendering can be achieved using other methods such as single pass stereo using vertex instancing or geometry shaders. The two-rendering-passes method is a slower but more convenient way to achieve stereo rendering.

Once the game is running, and resources are loaded, we update the projection matrix, once per rendering pass. Objects are slightly different from each view. Next, we set up the graphics rendering pipeline.

> **Note**
>
> See **Create and load DirectX graphic resources** for more information on how resources are loaded.

In this sample game, the renderer is designed to use a standard vertex layout across all objects. This simplifies the shader design, and allows for easy changes between shaders, independent of the objects' geometry.

GameRenderer::Render method

We set the Direct3D context to use an input vertex layout. Input-layout objects describe how vertex buffer data is streamed into the rendering pipeline.

Next, we set the Direct3D context to use the constant buffers defined earlier, which are used by the vertex shader pipeline stage and the pixel shader pipeline stage.

> **Note**
>
> See **Rendering framework II: Game rendering** for more information about definition of the constant buffers.

Because the same input layout and set of constant buffers is used for all shaders that are in the pipeline, it's set up once per frame.

C++/WinRT

```
void GameRenderer::Render()
{
    bool stereoEnabled{ m_deviceResources->GetStereoState() };

    auto d3dContext{ m_deviceResources->GetD3DDeviceContext() };
    auto d2dContext{ m_deviceResources->GetD2DDeviceContext() };

    int renderingPasses = 1;
    if (stereoEnabled)
    {
        renderingPasses = 2;
    }

    for (int i = 0; i < renderingPasses; i++)
    {
        // Iterate through the number of rendering passes to be completed.
        // 2 rendering passes if stereo is enabled.
        if (i > 0)
        {
            // Doing the Right Eye View.
            ID3D11RenderTargetView* const targets[1] = { m_deviceResources-
>GetBackBufferRenderTargetViewRight() };

            // Resets render targets to the screen.
            // OMSetRenderTargets binds 2 things to the device.
            // 1. Binds one render target atomically to the device.
            // 2. Binds the depth-stencil view, as returned by the
GetDepthStencilView method, to the device.
            // For more info, see
            // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-omsetrendertargets

            d3dContext->OMSetRenderTargets(1, targets, m_deviceResources-
>GetDepthStencilView());

            // Clears the depth stencil view.
            // A depth stencil view contains the format and buffer to hold
depth and stencil info.
            // For more info about depth stencil view, go to:
            // https://learn.microsoft.com/windows/uwp/graphics-
concepts/depth-stencil-view--dsv-
            // A depth buffer is used to store depth information to control
which areas of
            // polygons are rendered rather than hidden from view. To learn
more about a depth buffer,
            // go to: https://learn.microsoft.com/windows/uwp/graphics-
concepts/depth-buffers
            // A stencil buffer is used to mask pixels in an image, to
produce special effects.
            // The mask determines whether a pixel is drawn or not,
            // by setting the bit to a 1 or 0. To learn more about a stencil
buffer,
```

```
            // go to: https://learn.microsoft.com/windows/uwp/graphics-
concepts/stencil-buffers

            d3dContext->ClearDepthStencilView(m_deviceResources-
>GetDepthStencilView(), D3D11_CLEAR_DEPTH, 1.0f, 0);

            // Direct2D -- discussed later
            d2dContext->SetTarget(m_deviceResources-
>GetD2DTargetBitmapRight());
        }
        else
        {
            // Doing the Mono or Left Eye View.
            // As compared to the right eye:
            // m_deviceResources->GetBackBufferRenderTargetView instead of
GetBackBufferRenderTargetViewRight
            ID3D11RenderTargetView* const targets[1] = { m_deviceResources-
>GetBackBufferRenderTargetView() };

            // Same as the Right Eye View.
            d3dContext->OMSetRenderTargets(1, targets, m_deviceResources-
>GetDepthStencilView());
            d3dContext->ClearDepthStencilView(m_deviceResources-
>GetDepthStencilView(), D3D11_CLEAR_DEPTH, 1.0f, 0);

            // d2d -- Discussed later under Adding UI
            d2dContext->SetTarget(m_deviceResources->GetD2DTargetBitmap());
        }

        const float clearColor[4] = { 0.5f, 0.5f, 0.8f, 1.0f };

        // Only need to clear the background when not rendering the full 3D
scene since
        // the 3D world is a fully enclosed box and the dynamics prevents
the camera from
        // moving outside this space.
        if (i > 0)
        {
            // Doing the Right Eye View.
            d3dContext->ClearRenderTargetView(m_deviceResources-
>GetBackBufferRenderTargetViewRight(), clearColor);
        }
        else
        {
            // Doing the Mono or Left Eye View.
            d3dContext->ClearRenderTargetView(m_deviceResources-
>GetBackBufferRenderTargetView(), clearColor);
        }

        // Render the scene objects
        if (m_game != nullptr && m_gameResourcesLoaded &&
m_levelResourcesLoaded)
        {
            // This section is only used after the game state has been
initialized and all device
```

```
            // resources needed for the game have been created and
associated with the game objects.
            if (stereoEnabled)
            {
                // When doing stereo, it is necessary to update the
projection matrix once per rendering pass.

                auto orientation = m_deviceResources-
>GetOrientationTransform3D();

                ConstantBufferChangeOnResize changesOnResize;
                // Apply either a left or right eye projection, which is an
offset from the middle
                XMStoreFloat4x4(
                    &changesOnResize.projection,
                    XMMatrixMultiply(
                        XMMatrixTranspose(
                            i == 0 ?
                            m_game->GameCamera().LeftEyeProjection() :
                            m_game->GameCamera().RightEyeProjection()
                            ),
                        XMMatrixTranspose(XMLoadFloat4x4(&orientation))
                        )
                    );

                d3dContext->UpdateSubresource(
                    m_constantBufferChangeOnResize.get(),
                    0,
                    nullptr,
                    &changesOnResize,
                    0,
                    0
                    );
            }

            // Update variables that change once per frame.
            ConstantBufferChangesEveryFrame
constantBufferChangesEveryFrameValue;
            XMStoreFloat4x4(
                &constantBufferChangesEveryFrameValue.view,
                XMMatrixTranspose(m_game->GameCamera().View())
                );
            d3dContext->UpdateSubresource(
                m_constantBufferChangesEveryFrame.get(),
                0,
                nullptr,
                &constantBufferChangesEveryFrameValue,
                0,
                0
                );

            // Set up the graphics pipeline. This sample uses the same
InputLayout and set of
            // constant buffers for all shaders, so they only need to be set
once per frame.
```

```
            // For more info about the graphics or rendering pipeline, see
            //
https://learn.microsoft.com/windows/win32/direct3d11/overviews-direct3d-11-
graphics-pipeline

            // IASetInputLayout binds an input-layout object to the input-
assembler (IA) stage.
            // Input-layout objects describe how vertex buffer data is
streamed into the IA pipeline stage.
            // Set up the Direct3D context to use this vertex layout. For
more info, see
            // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-iasetinputlayout
            d3dContext->IASetInputLayout(m_vertexLayout.get());

            // VSSetConstantBuffers sets the constant buffers used by the
vertex shader pipeline stage.
            // Set up the Direct3D context to use these constant buffers.
For more info, see
            // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-vssetconstantbuffers

            ID3D11Buffer* constantBufferNeverChanges{
m_constantBufferNeverChanges.get() };
            d3dContext->VSSetConstantBuffers(0, 1,
&constantBufferNeverChanges);
            ID3D11Buffer* constantBufferChangeOnResize{
m_constantBufferChangeOnResize.get() };
            d3dContext->VSSetConstantBuffers(1, 1,
&constantBufferChangeOnResize);
            ID3D11Buffer* constantBufferChangesEveryFrame{
m_constantBufferChangesEveryFrame.get() };
            d3dContext->VSSetConstantBuffers(2, 1,
&constantBufferChangesEveryFrame);
            ID3D11Buffer* constantBufferChangesEveryPrim{
m_constantBufferChangesEveryPrim.get() };
            d3dContext->VSSetConstantBuffers(3, 1,
&constantBufferChangesEveryPrim);

            // Sets the constant buffers used by the pixel shader pipeline
stage.
            // For more info, see
            // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-pssetconstantbuffers

            d3dContext->PSSetConstantBuffers(2, 1,
&constantBufferChangesEveryFrame);
            d3dContext->PSSetConstantBuffers(3, 1,
&constantBufferChangesEveryPrim);
            ID3D11SamplerState* samplerLinear{ m_samplerLinear.get() };
            d3dContext->PSSetSamplers(0, 1, &samplerLinear);

            for (auto&& object : m_game->RenderObjects())
            {
                // The 3D object render method handles the rendering.
```

```
                // For more info, see Primitive rendering below.
                object->Render(d3dContext,
m_constantBufferChangesEveryPrim.get());
            }
        }

        // Start of 2D rendering
        ...
    }
}
```

Primitive rendering

When rendering the scene, you loop through all the objects that need to be rendered. The steps below are repeated for each object (primitive).

- Update the constant buffer (**m_constantBufferChangesEveryPrim**) with the model's world transformation matrix and material information.
- The **m_constantBufferChangesEveryPrim** contains parameters for each object. It includes the object-to-world transformation matrix as well as material properties such as color and specular exponent for lighting calculations.
- Set the Direct3D context to use the input vertex layout for the mesh object data to be streamed into the input-assembler (IA) stage of the rendering pipeline.
- Set the Direct3D context to use an index buffer in the IA stage. Provide the primitive info: type, data order.
- Submit a draw call to draw the indexed, non-instanced primitive. The **GameObject::Render** method updates the primitive constant buffer with the data specific to a given primitive. This results in a **DrawIndexed** call on the context to draw the geometry of that each primitive. Specifically, this draw call queues commands and data to the graphics processing unit (GPU), as parameterized by the constant buffer data. Each draw call executes the vertex shader one time per vertex, and then the pixel shader one time for every pixel of each triangle in the primitive. The textures are part of the state that the pixel shader uses to do the rendering.

Here are the reasons for using multiple constant buffers.

- The game uses multiple constant buffers, but it only needs to update these buffers one time per primitive. As mentioned earlier, constant buffers are like inputs to the shaders that run for each primitive. Some data is static (**m_constantBufferNeverChanges**); some data is constant over the frame (**m_constantBufferChangesEveryFrame**), such as the position of the camera; and

some data is specific to the primitive, such as its color and textures (**m_constantBufferChangesEveryPrim**).

- The game renderer separates these inputs into different constant buffers to optimize the memory bandwidth that the CPU and GPU use. This approach also helps to minimize the amount of data that the GPU needs to keep track of. The GPU has a big queue of commands, and each time the game calls **Draw**, that command is queued along with the data associated with it. When the game updates the primitive constant buffer and issues the next **Draw** command, the graphics driver adds this next command and the associated data to the queue. If the game draws 100 primitives, it could potentially have 100 copies of the constant buffer data in the queue. To minimize the amount of data the game is sending to the GPU, the game uses a separate primitive constant buffer that only contains the updates for each primitive.

GameObject::Render method

C++/WinRT

```
void GameObject::Render(
    _In_ ID3D11DeviceContext* context,
    _In_ ID3D11Buffer* primitiveConstantBuffer
    )
{
    if (!m_active || (m_mesh == nullptr) || (m_normalMaterial == nullptr))
    {
        return;
    }

    ConstantBufferChangesEveryPrim constantBuffer;

    // Put the model matrix info into a constant buffer, in world matrix.
    XMStoreFloat4x4(
        &constantBuffer.worldMatrix,
        XMMatrixTranspose(ModelMatrix())
        );

    // Check to see which material to use on the object.
    // If a collision (a hit) is detected, GameObject::Render checks the
current context, which
    // indicates whether the target has been hit by an ammo sphere. If the
target has been hit,
    // this method applies a hit material, which reverses the colors of the
rings of the target to
    // indicate a successful hit to the player. Otherwise, it applies the
default material
    // with the same method. In both cases, it sets the material by calling
Material::RenderSetup,
    // which sets the appropriate constants into the constant buffer. Then,
it calls
```

```
    // ID3D11DeviceContext::PSSetShaderResources to set the corresponding
texture resource for the
    // pixel shader, and ID3D11DeviceContext::VSSetShader and
ID3D11DeviceContext::PSSetShader
    // to set the vertex shader and pixel shader objects themselves,
respectively.

    if (m_hit && m_hitMaterial != nullptr)
    {
        m_hitMaterial->RenderSetup(context, &constantBuffer);
    }
    else
    {
        m_normalMaterial->RenderSetup(context, &constantBuffer);
    }

    // Update the primitive constant buffer with the object model's info.
    context->UpdateSubresource(primitiveConstantBuffer, 0, nullptr,
&constantBuffer, 0, 0);

    // Render the mesh.
    // See MeshObject::Render method below.
    m_mesh->Render(context);
}
```

MeshObject::Render method

C++/WinRT

```
void MeshObject::Render(_In_ ID3D11DeviceContext* context)
{
    // PNTVertex is a struct. stride provides us the size required for all
the mesh data
    // struct PNTVertex
    //{
    //  DirectX::XMFLOAT3 position;
    //  DirectX::XMFLOAT3 normal;
    //  DirectX::XMFLOAT2 textureCoordinate;
    //};
    uint32_t stride{ sizeof(PNTVertex) };
    uint32_t offset{ 0 };

    // Similar to the main render loop.
    // Input-layout objects describe how vertex buffer data is streamed into
the IA pipeline stage.
    ID3D11Buffer* vertexBuffer{ m_vertexBuffer.get() };
    context->IASetVertexBuffers(0, 1, &vertexBuffer, &stride, &offset);

    // IASetIndexBuffer binds an index buffer to the input-assembler stage.
    // For more info, see
    // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-iasetindexbuffer.
```

```
    context->IASetIndexBuffer(m_indexBuffer.get(), DXGI_FORMAT_R16_UINT, 0);

    // Binds information about the primitive type, and data order that
describes input data for the input assembler stage.
    // For more info, see
    // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-iasetprimitivetopology.
    context->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);

    // Draw indexed, non-instanced primitives. A draw API submits work to
the rendering pipeline.
    // For more info, see
    // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11devicecontext-drawindexed.
    context->DrawIndexed(m_indexCount, 0, 0);
}
```

DeviceResources::Present method

We call the **DeviceResources::Present** method to display the contents we've placed in the buffers.

We use the term swap chain for a collection of buffers that are used for displaying frames to the user. Each time an application presents a new frame for display, the first buffer in the swap chain takes the place of the displayed buffer. This process is called swapping or flipping. For more information, see Swap chains.

- The **IDXGISwapChain1** interface's **Present** method instructs DXGI to block until vertical synchronization (VSync) takes place, putting the application to sleep until the next VSync. This ensures that you don't waste any cycles rendering frames that will never be displayed to the screen.
- The **ID3D11DeviceContext3** interface's **DiscardView** method discards the contents of the render target. This is a valid operation only when the existing contents will be entirely overwritten. If dirty or scroll rects are used, then this call should be removed.

- Using the same **DiscardView** method, discard the contents of the depth-stencil.
- The **HandleDeviceLost** method is used to manage the scenario of the device being removed. If the device was removed either by a disconnection or a driver upgrade, then you must recreate all device resources. For more information, see Handle device removed scenarios in Direct3D 11.

Tip

To achieve a smooth frame rate, you must ensure that the amount of work to render a frame fits in the time between VSyncs.

C++/WinRT

```
// Present the contents of the swap chain to the screen.
void DX::DeviceResources::Present()
{
    // The first argument instructs DXGI to block until VSync, putting the
application
    // to sleep until the next VSync. This ensures we don't waste any cycles
rendering
    // frames that will never be displayed to the screen.
    HRESULT hr = m_swapChain->Present(1, 0);

    // Discard the contents of the render target.
    // This is a valid operation only when the existing contents will be
entirely
    // overwritten. If dirty or scroll rects are used, this call should be
removed.
    m_d3dContext->DiscardView(m_d3dRenderTargetView.get());

    // Discard the contents of the depth stencil.
    m_d3dContext->DiscardView(m_d3dDepthStencilView.get());

    // If the device was removed either by a disconnection or a driver
upgrade, we
    // must recreate all device resources.
    if (hr == DXGI_ERROR_DEVICE_REMOVED || hr == DXGI_ERROR_DEVICE_RESET)
    {
        HandleDeviceLost();
    }
    else
    {
        winrt::check_hresult(hr);
    }
}
```

Next steps

This topic explained how graphics is rendered on the display, and it provides a short description for some of the rendering terms used (below). Learn more about rendering in the Rendering framework II: Game rendering topic, and learn how to prepare the data needed before rendering.

Terms and concepts

Simple game scene

A simple game scene is made up of a few objects with several light sources.

An object's shape is defined by a set of X, Y, Z coordinates in space. The actual render location in the game world can be determined by applying a transformation matrix to the positional X, Y, Z coordinates. It may also have a set of texture coordinates—U and V—which specify how a material is applied to the object. This defines the surface properties of the object, and gives you the ability to see whether an object has a rough surface (like a tennis ball), or a smooth glossy surface (like a bowling ball).

Scene and object info is used by the rendering framework to recreate the scene frame by frame, making it come alive on your display monitor.

Rendering pipeline

The rendering pipeline is the process by which 3D scene info is translated to an image displayed on screen. In Direct3D 11, this pipeline is programmable. You can adapt the stages to support your rendering needs. Stages that feature common shader cores are programmable by using the HLSL programming language. It's also known as the *graphics rendering pipeline*, or simply *pipeline*.

To help you create this pipeline, you need to be familiar with these details.

- HLSL. We recommend the use of HLSL Shader Model 5.1 and above for UWP DirectX games.
- Shaders.
- Vertex shaders and pixel shaders.
- Shader stages.
- Various shader file formats.

For more information, see Understand the Direct3D 11 rendering pipeline and Graphics pipeline.

HLSL

HLSL is the high-level shader language for DirectX. Using HLSL, you can create C-like programmable shaders for the Direct3D pipeline. For more information, see HLSL.

Shaders

A shader can be thought of as a set of instructions that determine how the surface of an object appears when rendered. Those that are programmed using HLSL are known as HLSL shaders. Source code files for [HLSL])(#hlsl) shaders have the `.hlsl` file extension. These shaders can be compiled at build-time or at runtime, and set at runtime into the appropriate pipeline stage. A compiled shader object has a `.cso` file extension.

Direct3D 9 shaders can be designed using shader model 1, shader model 2 and shader model 3; Direct3D 10 shaders can be designed only on shader model 4. Direct3D 11 shaders can be designed on shader model 5. Direct3D 11.3 and Direct3D 12 can be designed on shader model 5.1, and Direct3D 12 can also be designed on shader model 6.

Vertex shaders and pixel shaders

Data enters the graphics pipeline as a stream of primitives, and is processed by various shaders such as the vertex shaders and pixel shaders.

Vertex shaders processes vertices, typically performing operations such as transformations, skinning, and lighting. Pixel shaders enables rich shading techniques such as per-pixel lighting and post-processing. It combines constant variables, texture data, interpolated per-vertex values, and other data to produce per-pixel outputs.

Shader stages

A sequence of these various shaders defined to process this stream of primitives is known as shader stages in a rendering pipeline. The actual stages depend on the version of Direct3D, but usually include the vertex, geometry, and pixel stages. There are also other stages, such as the hull and domain shaders for tessellation, and the compute shader. All these stages are completely programmable using HLSL. For more information, see Graphics pipeline.

Various shader file formats

Here are the shader code file extensions.

- A file with the `.hlsl` extension holds [HLSL])(#hlsl) source code.
- A file with the `.cso` extension holds a compiled shader object.
- A file with the `.h` extension is a header file, but in a shader code context, this header file defines a byte array that holds shader data.
- A file with the `.hlsli` extension contains the format of the constant buffers. In the sample game, the file is **Shaders** > **ConstantBuffers.hlsli**.

> **Note**
>
> You embed a shader either by loading a `.cso` file at runtime, or by adding a `.h` file in your executable code. But you wouldn't use both for the same shader.

Deeper understanding of DirectX

Direct3D 11 is a set of APIs that can help us to create graphics for graphics intensive applications such as games, where we want to have a good graphics card to process intensive computation. This section briefly explains the Direct3D 11 graphics programming concepts: resource, subresource, device, and device context.

Resource

You can think of resources (also known as device resources) as info about how to render an object, such as texture, position, or color. Resources provide data to the pipeline, and define what is rendered during your scene. Resources can be loaded from your game media, or created dynamically at run time.

A resource is, in fact, an area in memory that can be accessed by the Direct3D pipeline. In order for the pipeline to access memory efficiently, data that is provided to the pipeline (such as input geometry, shader resources, and textures) must be stored in a resource. There are two types of resources from which all Direct3D resources derive: a buffer or a texture. Up to 128 resources can be active for each pipeline stage. For more information, see Resources.

Subresource

The term subresource refers to a subset of a resource. Direct3D can reference an entire resource, or it can reference subsets of a resource. For more information, see Subresource.

Depth-stencil

A depth-stencil resource contains the format and buffer to hold depth and stencil information. It is created using a texture resource. For more information on how to create a depth-stencil resource, see Configuring Depth-Stencil Functionality. We access the depth-stencil resource through the depth-stencil view implemented using the ID3D11DepthStencilView interface.

Depth info tells us which areas of polygons are behind others, so that we can determine which are hidden. Stencil info tells us which pixels are masked. It can be used to produce special effects since it determines whether a pixel is drawn or not; sets the bit to a 1 or 0.

For more information, see Depth-stencil view, depth buffer, and stencil buffer.

Render target

A render target is a resource that we can write to at the end of a render pass. It is commonly created using the ID3D11Device::CreateRenderTargetView method using the swap chain back buffer (which is also a resource) as the input parameter.

Each render target should also have a corresponding depth-stencil view because when we use OMSetRenderTargets to set the render target before using it, it requires also a depth-stencil view. We access the render target resource through the render target view implemented using the ID3D11RenderTargetView interface.

Device

You can imagine a device as a way to allocate and destroy objects, render primitives, and communicate with the graphics card through the graphics driver.

For a more precise explanation, a Direct3D device is the rendering component of Direct3D. A device encapsulates and stores the rendering state, performs transformations and lighting operations, and rasterizes an image to a surface. For more information, see Devices

A device is represented by the **ID3D11Device** interface. In other words, the **ID3D11Device** interface represents a virtual display adapter, and is used to create resources that are owned by a device.

There are different versions of ID3D11Device. **ID3D11Device5** is the latest version, and adds new methods to those in **ID3D11Device4**. For more information on how Direct3D communicates with the underlying hardware, see Windows Device Driver Model (WDDM) architecture.

Each application must have at least one device; most applications create only one. Create a device for one of the hardware drivers installed on your machine by calling **D3D11CreateDevice** or **D3D11CreateDeviceAndSwapChain** and specifying the driver type with the **D3D_DRIVER_TYPE** flag. Each device can use one or more device contexts, depending on the functionality desired. For more information, see D3D11CreateDevice function.

Device context

A device context is used to set pipeline state, and generate rendering commands using the resources owned by a device.

Direct3D 11 implements two types of device contexts, one for immediate rendering and the other for deferred rendering; both contexts are represented with an ID3D11DeviceContext interface.

The **ID3D11DeviceContext** interfaces have different versions; **ID3D11DeviceContext4** adds new methods to those in **ID3D11DeviceContext3**.

ID3D11DeviceContext4 is introduced in the Windows 10 Creators Update, and is the latest version of the **ID3D11DeviceContext** interface. Applications targeting Windows 10 Creators Update and later should use this interface instead of earlier versions. For more information, see ID3D11DeviceContext4.

DX::DeviceResources

The **DX::DeviceResources** class is in the **DeviceResources.cpp/.h** files, and controls all of DirectX device resources.

Buffer

A buffer resource is a collection of fully typed data grouped into elements. You can use buffers to store a wide variety of data, including position vectors, normal vectors, texture coordinates in a vertex buffer, indexes in an index buffer, or device state. Buffer elements can include packed data values (such as **R8G8B8A8** surface values), single 8-bit integers, or four 32-bit floating point values.

There are three types of buffers available: vertex buffer, index buffer, and constant buffer.

Vertex buffer

Contains the vertex data used to define your geometry. Vertex data includes position coordinates, color data, texture coordinate data, normal data, and so on.

Index buffer

Contains integer offsets into vertex buffers and are used to render primitives more efficiently. An index buffer contains a sequential set of 16-bit or 32-bit indices; each

index is used to identify a vertex in a vertex buffer.

Constant buffer, or shader-constant buffer

Allows you to efficiently supply shader data to the pipeline. You can use constant buffers as inputs to the shaders that run for each primitive and store results of the stream-output stage of the rendering pipeline. Conceptually, a constant buffer looks just like a single-element vertex buffer.

Design and implementation of buffers

You can design buffers based on the data type, for example, like in our sample game, one buffer is created for static data, another for data that's constant over the frame, and another for data that's specific to a primitive.

All buffer types are encapsulated by the **ID3D11Buffer** interface and you can create a buffer resource by calling **ID3D11Device::CreateBuffer**. But a buffer must be bound to the pipeline before it can be accessed. Buffers can be bound to multiple pipeline stages simultaneously for reading. A buffer can also be bound to a single pipeline stage for writing; however, the same buffer cannot be bound for both reading and writing simultaneously.

You can bind buffers in these ways.

- To the input-assembler stage by calling **ID3D11DeviceContext** methods such as **ID3D11DeviceContext::IASetVertexBuffers** and **ID3D11DeviceContext::IASetIndexBuffer**.
- To the stream-output stage by calling **ID3D11DeviceContext::SOSetTargets**.
- To the shader stage by calling shader methods, such as **ID3D11DeviceContext::VSSetConstantBuffers**.

For more information, see Introduction to buffers in Direct3D 11.

DXGI

Microsoft DirectX Graphics Infrastructure (DXGI) is a subsystem that encapsulates some of the low-level tasks that are needed by Direct3D. Special care must be taken when using DXGI in a multithreaded application to ensure that deadlocks don't occur. For more info, see Multithreading and DXGI

Feature level

Feature level is a concept introduced in Direct3D 11 to handle the diversity of video cards in new and existing machines. A feature level is a well-defined set of graphics processing unit (GPU) functionality.

Each video card implements a certain level of DirectX functionality depending on the GPUs installed. In prior versions of Microsoft Direct3D, you could find out the version of Direct3D the video card implemented, and then program your application accordingly.

With feature level, when you create a device, you can attempt to create a device for the feature level that you want to request. If the device creation works, that feature level exists, if not, the hardware does not support that feature level. You can either try to recreate a device at a lower feature level, or you can choose to exit the application. For instance, the 12_0 feature level requires Direct3D 11.3 or Direct3D 12, and shader model 5.1. For more information, see Direct3D feature levels: Overview for each feature level.

Using feature levels, you can develop an application for Direct3D 9, Microsoft Direct3D 10, or Direct3D 11, and then run it on 9, 10, or 11 hardware (with some exceptions). For more information, see Direct3D feature levels.

Stereo rendering

Stereo rendering is used to enhance the illusion of depth. It uses two images, one from the left eye and the other from the right eye to display a scene on the display screen.

Mathematically, we apply a stereo projection matrix, which is a slight horizontal offset to the right and to the left, of the regular mono projection matrix to achieve this.

We did two rendering passes to achieve stereo rendering in this sample game.

- Bind to right render target, apply right projection, then draw the primitive object.
- Bind to left render target, apply left projection, then draw the primitive object.

Camera and coordinate space

The game has the code in place to update the world in its own coordinate system (sometimes called the world space or scene space). All objects, including the camera, are positioned and oriented in this space. For more information, see Coordinate systems.

A vertex shader does the heavy lifting of converting from the model coordinates to device coordinates with the following algorithm (where V is a vector and M is a matrix).

```
V(device) = V(model) x M(model-to-world) x M(world-to-view) x M(view-to-device)
```

- `M(model-to-world)` is a transformation matrix for model coordinates to world coordinates, also known as the World transform matrix. This is provided by the primitive.
- `M(world-to-view)` is a transformation matrix for world coordinates to view coordinates, also known as the View transform matrix.
  - This is provided by the view matrix of the camera. It's defined by the camera's position along with the look vectors (the *look at* vector that points directly into the scene from the camera, and the *look up* vector that is upwards perpendicular to it).
  - In the sample game, **m_viewMatrix** is the view transformation matrix, and is calculated using **Camera::SetViewParams**.
- `M(view-to-device)` is a transformation matrix for view coordinates to device coordinates, also known as the Projection transform matrix.
  - This is provided by the projection of the camera. It provides info about how much of that space is actually visible in the final scene. The field of view (FoV), aspect ratio, and clipping planes define the projection transform matrix.
  - In the sample game, **m_projectionMatrix** defines transformation to the projection coordinates, calculated using **Camera::SetProjParams** (For stereo projection, you use two projection matrices—one for each eye's view).

The shader code in `VertexShader.hlsl` is loaded with these vectors and matrices from the constant buffers, and performs this transformation for every vertex.

Coordinate transformation

Direct3D uses three transformations to change your 3D model coordinates into pixel coordinates (screen space). These transformations are world transform, view transform, and projection transform. For more info, see Transform overview.

World transform matrix

A world transform changes coordinates from model space, where vertices are defined relative to a model's local origin, to world space, where vertices are defined relative to an origin common to all the objects in a scene. In essence, the world transform places a model into the world; hence its name. For more information, see World transform.

View transform matrix

The view transform locates the viewer in world space, transforming vertices into camera space. In camera space, the camera, or viewer, is at the origin, looking in the positive z-

direction. For more info, go to View transform.

Projection transform matrix

The projection transform converts the viewing frustum to a cuboid shape. A viewing frustum is a 3D volume in a scene positioned relative to the viewport's camera. A viewport is a 2D rectangle into which a 3D scene is projected. For more information, see Viewports and clipping

Because the near end of the viewing frustum is smaller than the far end, this has the effect of expanding objects that are near to the camera; this is how perspective is applied to the scene. So objects that are closer to the player appear larger; objects that are further away appear smaller.

Mathematically, the projection transform is a matrix that is typically both a scale and a perspective projection. It functions like the lens of a camera. For more information, see Projection transform.

Sampler state

Sampler state determines how texture data is sampled using texture addressing modes, filtering, and level of detail. Sampling is done each time a texture pixel (or texel) is read from a texture.

A texture contains an array of texels. The position of each texel is denoted by `(u,v)`, where `u` is the width and `v` is the height, and is mapped between 0 and 1 based on the texture width and height. The resulting texture coordinates are used to address a texel when sampling a texture.

When texture coordinates are below 0 or above 1, the texture address mode defines how the texture coordinate addresses a texel location. For example, when using **TextureAddressMode.Clamp**, any coordinate outside the 0-1 range is clamped to a maximum value of 1, and minimum value of 0 before sampling.

If the texture is too large or too small for the polygon, then the texture is filtered to fit the space. A magnification filter enlarges a texture, a minification filter reduces the texture to fit into a smaller area. Texture magnification repeats the sample texel for one or more addresses which yields a blurrier image. Texture minification is more complicated because it requires combining more than one texel values into a single value. This can cause aliasing or jagged edges depending on the texture data. The most popular approach for minification is to use a mipmap. A mipmap is a multi-level texture. The size of each level is a power of 2 smaller than the previous level down to a 1x1

texture. When minification is used, a game chooses the mipmap level closest to the size that is needed at render time.

The BasicLoader class

BasicLoader is a simple loader class that provides support for loading shaders, textures, and meshes from files on disk. It provides both synchronous and asynchronous methods. In this sample game, the `BasicLoader.h/.cpp` files are found in the **Utilities** folder.

For more information, see Basic Loader.

Rendering framework II: Game rendering

Article • 10/20/2022

> **Note**
>
> This topic is part of the **Create a simple Universal Windows Platform (UWP) game with DirectX** tutorial series. The topic at that link sets the context for the series.

In Rendering framework I, we've covered how we take the scene info and present it to the display screen. Now, we'll take a step back and learn how to prepare the data for rendering.

> **Note**
>
> If you haven't downloaded the latest game code for this sample, go to **Direct3D sample game**. This sample is part of a large collection of UWP feature samples. For instructions on how to download the sample, see **Sample applications for Windows development**.

Objective

Quick recap on the objective. It is to understand how to set up a basic rendering framework to display the graphics output for a UWP DirectX game. We can loosely group them into these three steps.

1. Establish a connection to our graphics interface
2. Preparation: Create the resources we need to draw the graphics
3. Display the graphics: Render the frame

Rendering framework I: Intro to rendering explained how graphics are rendered, covering Steps 1 and 3.

This article explains how to set up other pieces of this framework and prepare the required data before rendering can happen, which is Step 2 of the process.

Design the renderer

The renderer is responsible for creating and maintaining all the D3D11 and D2D objects used to generate the game visuals. The **GameRenderer** class is the renderer for this sample game and is designed to meet the game's rendering needs.

These are some concepts you can use to help design the renderer for your game:

- Because Direct3D 11 APIs are defined as COM APIs, you must provide ComPtr references to the objects defined by these APIs. These objects are automatically freed when their last reference goes out of scope when the app terminates. For more information, see ComPtr. Example of these objects: constant buffers, shader objects - vertex shader, pixel shader, and shader resource objects.
- Constant buffers are defined in this class to hold various data needed for rendering.
  - Use multiple constant buffers with different frequencies to reduce the amount of data that must be sent to the GPU per frame. This sample separates constants into different buffers based on the frequency that they must be updated. This is a best practice for Direct3D programming.
  - In this sample game, 4 constant buffers are defined.

    1. **m_constantBufferNeverChanges** contains the lighting parameters. It's set one time in the **FinalizeCreateGameDeviceResources** method and never changes again.
    2. **m_constantBufferChangeOnResize** contains the projection matrix. The projection matrix is dependent on the size and aspect ratio of the window. It's set in **CreateWindowSizeDependentResources** and then updated after resources are loaded in the **FinalizeCreateGameDeviceResources** method. If rendering in 3D, it is also changed twice per frame.
    3. **m_constantBufferChangesEveryFrame** contains the view matrix. This matrix is dependent on the camera position and look direction (the normal to the projection) and changes one time per frame in the **Render** method. This was discussed earlier in **Rendering framework I: Intro to rendering**, under the **GameRenderer::Render** method.
    4. **m_constantBufferChangesEveryPrim** contains the model matrix and material properties of each primitive. The model matrix transforms vertices from local coordinates into world coordinates. These constants are specific to each primitive and are updated for every draw call. This was discussed earlier in **Rendering framework I: Intro to rendering**, under the Primitive rendering.

- Shader resource objects that hold textures for the primitives are also defined in this class.

    - Some textures are pre-defined (DDS is a file format that can be used to store compressed and uncompressed textures. DDS textures are used for the walls and floor of the world as well as the ammo spheres.)
    - In this sample game, shader resource objects are: **m_sphereTexture**, **m_cylinderTexture**, **m_ceilingTexture**, **m_floorTexture**, **m_wallsTexture**.
- Shader objects are defined in this class to compute our primitives and textures.
    - In this sample game, the shader objects are **m_vertexShader**, **m_vertexShaderFlat**, and **m_pixelShader**, **m_pixelShaderFlat**.
    - The vertex shader processes the primitives and the basic lighting, and the pixel shader (sometimes called a fragment shader) processes the textures and any per-pixel effects.
    - There are two versions of these shaders (regular and flat) for rendering different primitives. The reason we have different versions is that the flat versions are much simpler and don't do specular highlights or any per pixel lighting effects. These are used for the walls and make rendering faster on lower powered devices.

GameRenderer.h

Now let's look at the code in the sample game's renderer class object.

C++/WinRT

```
// Class handling the rendering of the game
class GameRenderer : public std::enable_shared_from_this<GameRenderer>
{
public:
    GameRenderer(std::shared_ptr<DX::DeviceResources> const&
deviceResources);

    void CreateDeviceDependentResources();
    void CreateWindowSizeDependentResources();
    void ReleaseDeviceDependentResources();
    void Render();
    // --- end of async related methods section

    winrt::Windows::Foundation::IAsyncAction
CreateGameDeviceResourcesAsync(_In_ std::shared_ptr<Simple3DGame> game);
    void FinalizeCreateGameDeviceResources();
    winrt::Windows::Foundation::IAsyncAction LoadLevelResourcesAsync();
    void FinalizeLoadLevelResources();

    Simple3DGameDX::IGameUIControl* GameUIControl() { return
&m_gameInfoOverlay; };

    DirectX::XMFLOAT2 GameInfoOverlayUpperLeft()
    {
```

```
        return DirectX::XMFLOAT2(m_gameInfoOverlayRect.left,
m_gameInfoOverlayRect.top);
    };
    DirectX::XMFLOAT2 GameInfoOverlayLowerRight()
    {
        return DirectX::XMFLOAT2(m_gameInfoOverlayRect.right,
m_gameInfoOverlayRect.bottom);
    };
    bool GameInfoOverlayVisible() { return m_gameInfoOverlay.Visible(); }
    // --- end of rendering overlay section
...
private:
    // Cached pointer to device resources.
    std::shared_ptr<DX::DeviceResources>        m_deviceResources;

    ...

    // Shader resource objects
    winrt::com_ptr<ID3D11ShaderResourceView>    m_sphereTexture;
    winrt::com_ptr<ID3D11ShaderResourceView>    m_cylinderTexture;
    winrt::com_ptr<ID3D11ShaderResourceView>    m_ceilingTexture;
    winrt::com_ptr<ID3D11ShaderResourceView>    m_floorTexture;
    winrt::com_ptr<ID3D11ShaderResourceView>    m_wallsTexture;

    // Constant buffers
    winrt::com_ptr<ID3D11Buffer>
m_constantBufferNeverChanges;
    winrt::com_ptr<ID3D11Buffer>
m_constantBufferChangeOnResize;
    winrt::com_ptr<ID3D11Buffer>
m_constantBufferChangesEveryFrame;
    winrt::com_ptr<ID3D11Buffer>
m_constantBufferChangesEveryPrim;

    // Texture sampler
    winrt::com_ptr<ID3D11SamplerState>          m_samplerLinear;

    // Shader objects: Vertex shaders and pixel shaders
    winrt::com_ptr<ID3D11VertexShader>          m_vertexShader;
    winrt::com_ptr<ID3D11VertexShader>          m_vertexShaderFlat;
    winrt::com_ptr<ID3D11PixelShader>           m_pixelShader;
    winrt::com_ptr<ID3D11PixelShader>           m_pixelShaderFlat;
    winrt::com_ptr<ID3D11InputLayout>           m_vertexLayout;
};
```

Constructor

Next, let's examine the sample game's **GameRenderer** constructor and compare it with the **Sample3DSceneRenderer** constructor provided in the DirectX 11 App template.

C++/WinRT

```
// Constructor method of the main rendering class object
GameRenderer::GameRenderer(std::shared_ptr<DX::DeviceResources> const&
deviceResources) : ...
    m_gameInfoOverlay(deviceResources),
    m_gameHud(deviceResources, L"Windows platform samples", L"DirectX first-
person game sample")
{
    // m_gameInfoOverlay is a GameHud object to render text in the top left
corner of the screen.
    // m_gameHud is Game info rendered as an overlay on the top-right corner
of the screen,
    // for example hits, shots, and time.

    CreateDeviceDependentResources();
    CreateWindowSizeDependentResources();
}
```

Create and load DirectX graphic resources

In the sample game (and in Visual Studio's **DirectX 11 App (Universal Windows)** template), creating and loading game resources is implemented using these two methods that are called from **GameRenderer** constructor:

- CreateDeviceDependentResources
- CreateWindowSizeDependentResources

CreateDeviceDependentResources method

In the DirectX 11 App template, this method is used to load vertex and pixel shader asynchronously, create the shader and constant buffer, create a mesh with vertices that contain position and color info.

In the sample game, these operations of the scene objects are instead split among the CreateGameDeviceResourcesAsync and FinalizeCreateGameDeviceResources methods.

For this sample game, what goes into this method?

- Instantiated variables (**m_gameResourcesLoaded** = false and **m_levelResourcesLoaded** = false) that indicate whether resources have been loaded before moving forward to render, since we're loading them asynchronously.
- Since HUD and overlay rendering are in separate class objects, call **GameHud::CreateDeviceDependentResources** and **GameInfoOverlay::CreateDeviceDependentResources** methods here.

Here's the code for **GameRenderer::CreateDeviceDependentResources**.

C++/WinRT

```
// This method is called in GameRenderer constructor when it's created in
GameMain constructor.
void GameRenderer::CreateDeviceDependentResources()
{
    // instantiate variables that indicate whether resources were loaded.
    m_gameResourcesLoaded = false;
    m_levelResourcesLoaded = false;

    // game HUD and overlay are design as separate class objects.
    m_gameHud.CreateDeviceDependentResources();
    m_gameInfoOverlay.CreateDeviceDependentResources();
}
```

Below is a list of the methods that are used to create and load resources.

- CreateDeviceDependentResources
  - CreateGameDeviceResourcesAsync (Added)
  - FinalizeCreateGameDeviceResources (Added)
- CreateWindowSizeDependentResources

Before diving into the other methods that are used to create and load resources, let's first create the renderer and see how it fits into the game loop.

Create the renderer

The **GameRenderer** is created in the **GameMain**'s constructor. It also calls the two other methods, **CreateGameDeviceResourcesAsync** and **FinalizeCreateGameDeviceResources** that are added to help create and load resources.

C++/WinRT

```
GameMain::GameMain(std::shared_ptr<DX::DeviceResources> const&
deviceResources) : ...
{
    m_deviceResources->RegisterDeviceNotify(this);

    // Creation of GameRenderer
    m_renderer = std::make_shared<GameRenderer>(m_deviceResources);

    ...

    ConstructInBackground();
}
```

```
winrt::fire_and_forget GameMain::ConstructInBackground()
{
    ...

    // Asynchronously initialize the game class and load the renderer device
resources.
    // By doing all this asynchronously, the game gets to its main loop more
quickly
    // and in parallel all the necessary resources are loaded on other
threads.
    m_game->Initialize(m_controller, m_renderer);

    co_await m_renderer->CreateGameDeviceResourcesAsync(m_game);

    // The finalize code needs to run in the same thread context
    // as the m_renderer object was created because the D3D device context
    // can ONLY be accessed on a single thread.
    // co_await of an IAsyncAction resumes in the same thread context.
    m_renderer->FinalizeCreateGameDeviceResources();

    InitializeGameState();

    ...
}
```

CreateGameDeviceResourcesAsync method

CreateGameDeviceResourcesAsync is called from the **GameMain** constructor method in the **create_task** loop since we're loading game resources asynchronously.

CreateDeviceResourcesAsync is a method that runs as a separate set of async tasks to load the game resources. Because it's expected to run on a separate thread, it only has access to the Direct3D 11 device methods (those defined on **ID3D11Device**) and not the device context methods (the methods defined on **ID3D11DeviceContext**), so it does not perform any rendering.

FinalizeCreateGameDeviceResources method runs on the main thread and does have access to the Direct3D 11 device context methods.

In principle:

- Use only **ID3D11Device** methods in **CreateGameDeviceResourcesAsync** because they are free-threaded, which means that they are able to run on any thread. It is also expected that they do not run on the same thread as the one **GameRenderer** was created on.
- Do not use methods in **ID3D11DeviceContext** here because they need to run on a single thread and on the same thread as **GameRenderer**.

- Use this method to create constant buffers.
- Use this method to load textures (like the .dds files) and shader info (like the .cso files) into the shaders.

This method is used to:

- Create the 4 constant buffers: **m_constantBufferNeverChanges**, **m_constantBufferChangeOnResize**, **m_constantBufferChangesEveryFrame**, **m_constantBufferChangesEveryPrim**
- Create a sampler-state object that encapsulates sampling information for a texture
- Create a task group that contains all async tasks created by the method. It waits for the completion of all these async tasks, and then calls **FinalizeCreateGameDeviceResources**.
- Create a loader using Basic Loader. Add the loader's async loading operations as tasks into the task group created earlier.
- Methods like **BasicLoader::LoadShaderAsync** and **BasicLoader::LoadTextureAsync** are used to load:
  - compiled shader objects (VertextShader.cso, VertexShaderFlat.cso, PixelShader.cso, and PixelShaderFlat.cso). For more info, go to Various shader file formats.
  - game specific textures (Assets\seafloor.dds, metal_texture.dds, cellceiling.dds, cellfloor.dds, cellwall.dds).

C++/WinRT

```
IAsyncAction GameRenderer::CreateGameDeviceResourcesAsync(_In_
std::shared_ptr<Simple3DGame> game)
{
    auto lifetime = shared_from_this();

    // Create the device dependent game resources.
    // Only the d3dDevice is used in this method. It is expected
    // to not run on the same thread as the GameRenderer was created.
    // Create methods on the d3dDevice are free-threaded and are safe while
any methods
    // in the d3dContext should only be used on a single thread and handled
    // in the FinalizeCreateGameDeviceResources method.
    m_game = game;

    auto d3dDevice = m_deviceResources->GetD3DDevice();

    // Define D3D11_BUFFER_DESC. See
    // https://learn.microsoft.com/windows/win32/api/d3d11/ns-d3d11-
d3d11_buffer_desc
    D3D11_BUFFER_DESC bd;
    ZeroMemory(&bd, sizeof(bd));

    // Create the constant buffers.
```

```
    bd.Usage = D3D11_USAGE_DEFAULT;
    ...

    // Create the constant buffers: m_constantBufferNeverChanges,
m_constantBufferChangeOnResize,
    // m_constantBufferChangesEveryFrame, m_constantBufferChangesEveryPrim
    // CreateBuffer is used to create one of these buffers: vertex buffer,
index buffer, or
    // shader-constant buffer. For CreateBuffer API ref info, see
    // https://learn.microsoft.com/windows/win32/api/d3d11/nf-d3d11-
id3d11device-createbuffer.
    winrt::check_hresult(
        d3dDevice->CreateBuffer(&bd, nullptr,
m_constantBufferNeverChanges.put())
        );

    ...

    // Define D3D11_SAMPLER_DESC. For API ref, see
    // https://learn.microsoft.com/windows/win32/api/d3d11/ns-d3d11-
d3d11_sampler_desc.
    D3D11_SAMPLER_DESC sampDesc;

    // ZeroMemory fills a block of memory with zeros. For API ref, see
    // https://learn.microsoft.com/previous-
versions/windows/desktop/legacy/aa366920(v=vs.85).
    ZeroMemory(&sampDesc, sizeof(sampDesc));

    sampDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_LINEAR;
    sampDesc.AddressU = D3D11_TEXTURE_ADDRESS_WRAP;
    sampDesc.AddressV = D3D11_TEXTURE_ADDRESS_WRAP;
    ...

    // Create a sampler-state object that encapsulates sampling information
for a texture.
    // The sampler-state interface holds a description for sampler state
that you can bind to any
    // shader stage of the pipeline for reference by texture sample
operations.
    winrt::check_hresult(
        d3dDevice->CreateSamplerState(&sampDesc, m_samplerLinear.put())
        );

    // Start the async tasks to load the shaders and textures.

    // Load compiled shader objects (VertextShader.cso,
VertexShaderFlat.cso, PixelShader.cso, and PixelShaderFlat.cso).
    // The BasicLoader class is used to convert and load common graphics
resources, such as meshes, textures,
    // and various shader objects into the constant buffers. For more info,
see
    // https://learn.microsoft.com/windows/uwp/gaming/complete-code-for-
basicloader.
    BasicLoader loader{ d3dDevice };
```

```
    std::vector<IAsyncAction> tasks;

    uint32_t numElements = ARRAYSIZE(PNTVertexLayout);

    // Load shaders asynchronously with the shader and pixel data using the
    // BasicLoader::LoadShaderAsync method. Push these method calls into a
list of tasks.
    tasks.push_back(loader.LoadShaderAsync(L"VertexShader.cso",
PNTVertexLayout, numElements, m_vertexShader.put(), m_vertexLayout.put()));
    tasks.push_back(loader.LoadShaderAsync(L"VertexShaderFlat.cso", nullptr,
numElements, m_vertexShaderFlat.put(), nullptr));
    tasks.push_back(loader.LoadShaderAsync(L"PixelShader.cso",
m_pixelShader.put()));
    tasks.push_back(loader.LoadShaderAsync(L"PixelShaderFlat.cso",
m_pixelShaderFlat.put()));

    // Make sure the previous versions if any of the textures are released.
    m_sphereTexture = nullptr;
    ...

    // Load Game specific textures (Assets\\seafloor.dds, metal_texture.dds,
cellceiling.dds,
    // cellfloor.dds, cellwall.dds).
    // Push these method calls also into a list of tasks.
    tasks.push_back(loader.LoadTextureAsync(L"Assets\\seafloor.dds",
nullptr, m_sphereTexture.put()));
    ...

    // Simulate loading additional resources by introducing a delay.
    tasks.push_back([]() -> IAsyncAction { co_await
winrt::resume_after(GameConstants::InitialLoadingDelay); }());

    // Returns when all the async tasks for loading the shader and texture
assets have completed.
    for (auto&& task : tasks)
    {
        co_await task;
    }
}
```

FinalizeCreateGameDeviceResources method

FinalizeCreateGameDeviceResources method is called after all the load resources tasks that are in the **CreateGameDeviceResourcesAsync** method completes.

- Initialize constantBufferNeverChanges with the light positions and color. Loads the initial data into the constant buffers with a device context method call to **ID3D11DeviceContext::UpdateSubresource**.
- Since asynchronously loaded resources have completed loading, it's time to associate them with the appropriate game objects.

- For each game object, create the mesh and the material using the textures that have been loaded. Then associate the mesh and material to the game object.
- For the targets game object, the texture which is composed of concentric colored rings, with a numeric value on the top, is not loaded from a texture file. Instead, it's procedurally generated using the code in **TargetTexture.cpp**. The **TargetTexture** class creates the necessary resources to draw the texture into an off screen resource at initialization time. The resulting texture is then associated with the appropriate target game objects.

FinalizeCreateGameDeviceResources and CreateWindowSizeDependentResources share similar portions of code for these:

- Use **SetProjParams** to ensure that the camera has the right projection matrix. For more info, go to Camera and coordinate space.
- Handle screen rotation by post multiplying the 3D rotation matrix to the camera's projection matrix. Then update the **ConstantBufferChangeOnResize** constant buffer with the resulting projection matrix.
- Set the **m_gameResourcesLoaded Boolean** global variable to indicate that the resources are now loaded in the buffers, ready for the next step. Recall that we first initialized this variable as **FALSE** in the **GameRenderer**'s constructor method, through the **GameRenderer::CreateDeviceDependentResources** method.
- When this **m_gameResourcesLoaded** is **TRUE**, rendering of the scene objects can take place. This was covered in the **Rendering framework I: Intro to rendering** article, under GameRenderer::Render method.

C++/WinRT

```
// This method is called from the GameMain constructor.
// Make sure that 2D rendering is occurring on the same thread as the main
rendering.
void GameRenderer::FinalizeCreateGameDeviceResources()
{
    // All asynchronously loaded resources have completed loading.
    // Now associate all the resources with the appropriate game objects.
    // This method is expected to run in the same thread as the GameRenderer
    // was created. All work will happen behind the "Loading ..." screen
after the
    // main loop has been entered.

    // Initialize the Constant buffer with the light positions
    // These are handled here to ensure that the d3dContext is only
    // used in one thread.

    auto d3dDevice = m_deviceResources->GetD3DDevice();

    ConstantBufferNeverChanges constantBufferNeverChanges;
    constantBufferNeverChanges.lightPosition[0] = XMFLOAT4(3.5f, 2.5f, 5.5f,
```

```
1.0f);
    ...
    constantBufferNeverChanges.lightColor = XMFLOAT4(0.25f, 0.25f, 0.25f,
1.0f);

    // CPU copies data from memory (constantBufferNeverChanges) to a
subresource
    // created in non-mappable memory (m_constantBufferNeverChanges) which
was created in the earlier
    // CreateGameDeviceResourcesAsync method. For UpdateSubresource API ref
info,
    // go to:
https://msdn.microsoft.com/library/windows/desktop/ff476486.aspx
    // To learn more about what a subresource is, go to:
    // https://msdn.microsoft.com/library/windows/desktop/ff476901.aspx

    m_deviceResources->GetD3DDeviceContext()->UpdateSubresource(
        m_constantBufferNeverChanges.get(),
        0,
        nullptr,
        &constantBufferNeverChanges,
        0,
        0
        );

    // For the objects that function as targets, they have two unique
generated textures.
    // One version is used to show that they have never been hit and the
other is
    // used to show that they have been hit.
    // TargetTexture is a helper class to procedurally generate textures for
game
    // targets. The class creates the necessary resources to draw the
texture into
    // an off screen resource at initialization time.

    TargetTexture textureGenerator(
        d3dDevice,
        m_deviceResources->GetD2DFactory(),
        m_deviceResources->GetDWriteFactory(),
        m_deviceResources->GetD2DDeviceContext()
        );

    // CylinderMesh is a class derived from MeshObject and creates a
ID3D11Buffer of
    // vertices and indices to represent a canonical cylinder (capped at
    // both ends) that is positioned at the origin with a radius of 1.0,
    // a height of 1.0 and with its axis in the +Z direction.
    // In the game sample, there are various types of mesh types:
    // CylinderMesh (vertical rods), SphereMesh (balls that the player
shoots),
    // FaceMesh (target objects), and WorldMesh (Floors and ceilings that
define the enclosed area)

    auto cylinderMesh = std::make_shared<CylinderMesh>(d3dDevice,
```

```
(uint16_t)26);
    ...

    // The Material class maintains the properties that represent how an
object will
    // look when it is rendered.  This includes the color of the object, the
    // texture used to render the object, and the vertex and pixel shader
that
    // should be used for rendering.

    auto cylinderMaterial = std::make_shared<Material>(
        XMFLOAT4(0.8f, 0.8f, 0.8f, .5f),
        XMFLOAT4(0.8f, 0.8f, 0.8f, .5f),
        XMFLOAT4(1.0f, 1.0f, 1.0f, 1.0f),
        15.0f,
        m_cylinderTexture.get(),
        m_vertexShader.get(),
        m_pixelShader.get()
        );

    ...

    // Attach the textures to the appropriate game objects.
    // We'll loop through all the objects that need to be rendered.
    for (auto&& object : m_game->RenderObjects())
    {
        if (object->TargetId() == GameConstants::WorldFloorId)
        {
            // Assign a normal material for the floor object.
            // This normal material uses the floor texture (cellfloor.dds)
that was loaded asynchronously from
            // the Assets folder using BasicLoader::LoadTextureAsync method
in the earlier
            // CreateGameDeviceResourcesAsync loop

            object->NormalMaterial(
                std::make_shared<Material>(
                    XMFLOAT4(0.5f, 0.5f, 0.5f, 1.0f),
                    XMFLOAT4(0.8f, 0.8f, 0.8f, 1.0f),
                    XMFLOAT4(0.3f, 0.3f, 0.3f, 1.0f),
                    150.0f,
                    m_floorTexture.get(),
                    m_vertexShaderFlat.get(),
                    m_pixelShaderFlat.get()
                    )
                );
            // Creates a mesh object called WorldFloorMesh and assign it to
the floor object.
            object->Mesh(std::make_shared<WorldFloorMesh>(d3dDevice));
        }
        ...
        else if (auto cylinder = dynamic_cast<Cylinder*>(object.get()))
        {
            cylinder->Mesh(cylinderMesh);
            cylinder->NormalMaterial(cylinderMaterial);
```

```
        }
        else if (auto target = dynamic_cast<Face*>(object.get()))
        {
            const int bufferLength = 16;
            wchar_t str[bufferLength];
            int len = swprintf_s(str, bufferLength, L"%d", target-
>TargetId());
            auto string{ winrt::hstring(str, len) };

            winrt::com_ptr<ID3D11ShaderResourceView> texture;
            textureGenerator.CreateTextureResourceView(string,
texture.put());
            target->NormalMaterial(
                std::make_shared<Material>(
                    XMFLOAT4(0.8f, 0.8f, 0.8f, 0.5f),
                    XMFLOAT4(0.8f, 0.8f, 0.8f, 0.5f),
                    XMFLOAT4(0.3f, 0.3f, 0.3f, 1.0f),
                    5.0f,
                    texture.get(),
                    m_vertexShader.get(),
                    m_pixelShader.get()
                    )
                );

            texture = nullptr;
            textureGenerator.CreateHitTextureResourceView(string,
texture.put());
            target->HitMaterial(
                std::make_shared<Material>(
                    XMFLOAT4(0.8f, 0.8f, 0.8f, 0.5f),
                    XMFLOAT4(0.8f, 0.8f, 0.8f, 0.5f),
                    XMFLOAT4(0.3f, 0.3f, 0.3f, 1.0f),
                    5.0f,
                    texture.get(),
                    m_vertexShader.get(),
                    m_pixelShader.get()
                    )
                );

            target->Mesh(targetMesh);
        }
        ...
    }

    // The SetProjParams method calculates the projection matrix based on
input params and
    // ensures that the camera has been initialized with the right
projection
    // matrix.
    // The camera is not created at the time the first window resize event
occurs.

    auto renderTargetSize = m_deviceResources->GetRenderTargetSize();
    m_game->GameCamera().SetProjParams(
        XM_PI / 2,
```

```
        renderTargetSize.Width / renderTargetSize.Height,
        0.01f,
        100.0f
        );

    // Make sure that the correct projection matrix is set in the
ConstantBufferChangeOnResize buffer.

    // Get the 3D rotation transform matrix. We are handling screen
rotations directly to eliminate an unaligned
    // fullscreen copy. So it is necessary to post multiply the 3D rotation
matrix to the camera's projection matrix
    // to get the projection matrix that we need.

    auto orientation = m_deviceResources->GetOrientationTransform3D();

    ConstantBufferChangeOnResize changesOnResize;

    // The matrices are transposed due to the shader code expecting the
matrices in the opposite
    // row/column order from the DirectX math library.

    // XMStoreFloat4x4 takes a matrix and writes the components out to
sixteen single-precision floating-point values at the given address.
    // The most significant component of the first row vector is written to
the first four bytes of the address,
    // followed by the second most significant component of the first row,
and so on. The second row is then written out in a
    // like manner to memory beginning at byte 16, followed by the third row
to memory beginning at byte 32, and finally
    // the fourth row to memory beginning at byte 48. For more API ref info,
go to:
    //
https://msdn.microsoft.com/library/windows/desktop/microsoft.directx_sdk.sto
ring.xmstorefloat4x4.aspx

    XMStoreFloat4x4(
        &changesOnResize.projection,
        XMMatrixMultiply(
            XMMatrixTranspose(m_game->GameCamera().Projection()),
            XMMatrixTranspose(XMLoadFloat4x4(&orientation))
            )
        );

    // UpdateSubresource method instructs CPU to copy data from memory
(changesOnResize) to a subresource
    // created in non-mappable memory (m_constantBufferChangeOnResize )
which was created in the earlier
    // CreateGameDeviceResourcesAsync method.

    m_deviceResources->GetD3DDeviceContext()->UpdateSubresource(
        m_constantBufferChangeOnResize.get(),
        0,
        nullptr,
        &changesOnResize,
```

```
        0,
        0
        );

    // Finally we set the m_gameResourcesLoaded as TRUE, so we can start
rendering.
    m_gameResourcesLoaded = true;
}
```

CreateWindowSizeDependentResource method

CreateWindowSizeDependentResources methods are called every time the window size, orientation, stereo-enabled rendering, or resolution changes. In the sample game, it updates the projection matrix in **ConstantBufferChangeOnResize**.

Window size resources are updated in this manner:

- The App framework gets one of several possible events indicating a change in the window state.
- Your main game loop is then informed about the event and calls **CreateWindowSizeDependentResources** on the main class (**GameMain**) instance, which then calls the **CreateWindowSizeDependentResources** implementation in the game renderer (**GameRenderer**) class.
- The primary job of this method is to make sure the visuals don't become confused or invalid because of a change in window properties.

For this sample game, a number of method calls are the same as the FinalizeCreateGameDeviceResources method. For code walkthrough, go to the previous section.

The game HUD and overlay window size rendering adjustments is covered under Add a user interface.

C++/WinRT

```
// Initializes view parameters when the window size changes.
void GameRenderer::CreateWindowSizeDependentResources()
{
    // Game HUD and overlay window size rendering adjustments are done here
    // but they'll be covered in the UI section instead.

    m_gameHud.CreateWindowSizeDependentResources();

    ...

    auto d3dContext = m_deviceResources->GetD3DDeviceContext();
    // In Sample3DSceneRenderer::CreateWindowSizeDependentResources, we had:
```

```
    // Size outputSize = m_deviceResources->GetOutputSize();

    auto renderTargetSize = m_deviceResources->GetRenderTargetSize();

    ...


m_gameInfoOverlay.CreateWindowSizeDependentResources(m_gameInfoOverlaySize);

    if (m_game != nullptr)
    {
        // Similar operations as the last section of
FinalizeCreateGameDeviceResources method
        m_game->GameCamera().SetProjParams(
            XM_PI / 2, renderTargetSize.Width / renderTargetSize.Height,
            0.01f,
            100.0f
            );

        XMFLOAT4X4 orientation = m_deviceResources-
>GetOrientationTransform3D();

        ConstantBufferChangeOnResize changesOnResize;
        XMStoreFloat4x4(
            &changesOnResize.projection,
            XMMatrixMultiply(
                XMMatrixTranspose(m_game->GameCamera().Projection()),
                XMMatrixTranspose(XMLoadFloat4x4(&orientation))
                )
            );

        d3dContext->UpdateSubresource(
            m_constantBufferChangeOnResize.get(),
            0,
            nullptr,
            &changesOnResize,
            0,
            0
            );
    }
}
```

Next steps

This is the basic process for implementing the graphics rendering framework of a game. The larger your game, the more abstractions you would have to put in place to handle hierarchies of object types and animation behaviors. You need to implement more complex methods for loading and managing assets such as meshes and textures. Next, let's learn how to add a user interface.

Add a user interface

Article • 10/20/2022

> **Note**
>
> This topic is part of the Create a simple Universal Windows Platform (UWP) game with DirectX tutorial series. The topic at that link sets the context for the series.

Now that our game has its 3D visuals in place, it's time to focus on adding some 2D elements so that the game can provide feedback about game state to the player. This can be accomplished by adding simple menu options and heads-up display components on top of the 3-D graphics pipeline output.

> **Note**
>
> If you haven't downloaded the latest game code for this sample, go to Direct3D sample game. This sample is part of a large collection of UWP feature samples. For instructions on how to download the sample, see Sample applications for Windows development.

Objective

Using Direct2D, add a number of user interface graphics and behaviors to our UWP DirectX game including:

- Heads-up display, including move-look controller boundry rectangles
- Game state menus

The user interface overlay

While there are many ways to display text and user interface elements in a DirectX game, we're going to focus on using Direct2D. We'll also be using DirectWrite for the text elements.

Direct2D is a set of 2D drawing APIs used to draw pixel-based primitives and effects. When starting out with Direct2D, it's best to keep things simple. Complex layouts and interface behaviors need time and planning. If your game requires a complex user

interface, like those found in simulation and strategy games, consider using XAML instead.

> ⓘ **Note**
>
> For info about developing a user interface with XAML in a UWP DirectX game, see Extending the sample game.

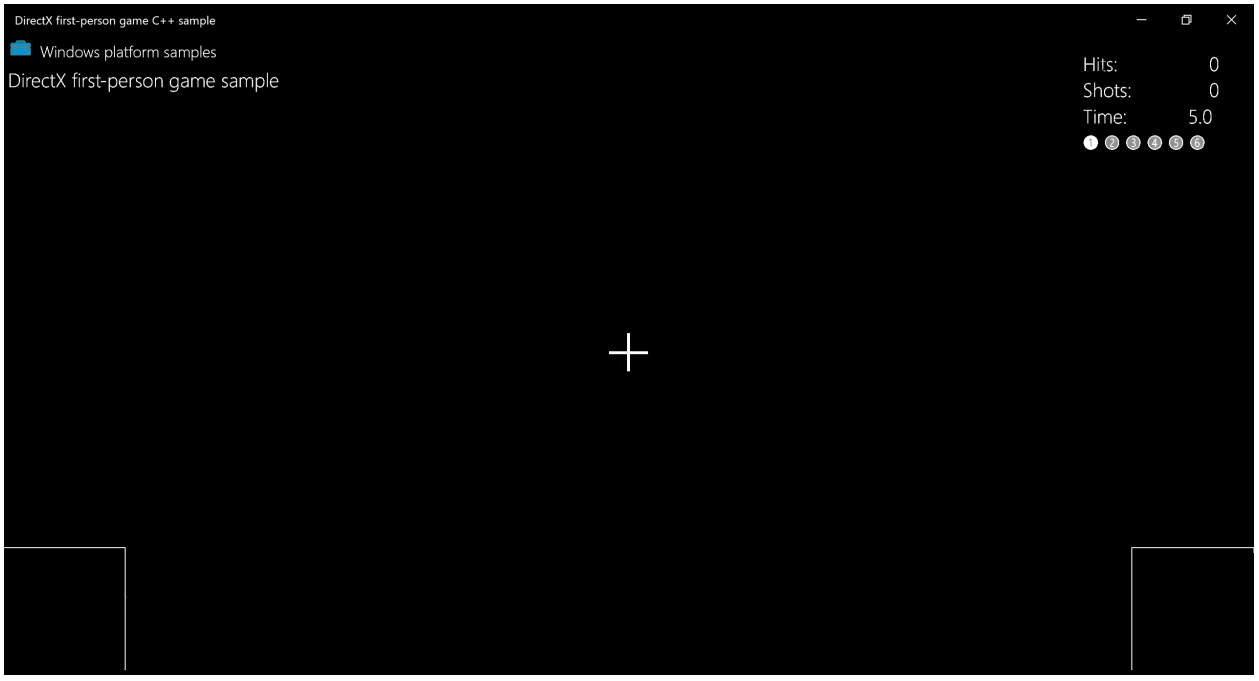
Direct2D isn't specifically designed for user interfaces or layouts like HTML and XAML. It doesn't provide user interface components like lists, boxes, or buttons. It also doesn't provide layout components like divs, tables, or grids.

For this sample game we have two major UI components.

1. A heads-up display for the score and in-game controls.
2. An overlay used to display game state text and options such as pause info and level start options.

Using Direct2D for a heads-up display

The following image shows the in-game heads-up display for the sample. It's simple and uncluttered, allowing the player to focus on navigating the 3D world and shooting targets. A good interface or heads-up display must never complicate the ability of the player to process and react to the events in the game.



The overlay consists of the following basic primitives.

- DirectWrite text in the upper-right corner that informs the player of

    - Successful hits
    - Number of shots the player has made
    - Time remaining in the level
    - Current level number
- Two intersecting line segments used to form a cross hair
- Two rectangles at the bottom corners for the move-look controller boundries.

The in-game heads-up display state of the overlay is drawn in the **GameHud::Render** method of the **GameHud** class. Within this method, the Direct2D overlay that represents our UI is updated to reflect the changes in the number of hits, time remaining, and level number.

If the game has been initialized, we add `TotalHits()`, `TotalShots()`, and `TimeRemaining()` to a **swprintf_s** buffer and specify the print format. We can then draw it using the **DrawText** method. We do the same for the current level indicator, drawing empty numbers to show uncompleted levels like ①, and filled numbers like ❶ to show that the specific level was completed.

The following code snippet walks through the **GameHud::Render** method's process for

- Creating a Bitmap using **ID2D1RenderTarget::DrawBitmap **
- Sectioning off UI areas into rectangles using **D2D1::RectF**
- Using **DrawText** to make text elements

C++/WinRT

```
void GameHud::Render(_In_ std::shared_ptr<Simple3DGame> const& game)
{
    auto d2dContext = m_deviceResources->GetD2DDeviceContext();
    auto windowBounds = m_deviceResources->GetLogicalSize();

    if (m_showTitle)
    {
        d2dContext->DrawBitmap(
            m_logoBitmap.get(),
            D2D1::RectF(
                GameUIConstants::Margin,
                GameUIConstants::Margin,
                m_logoSize.width + GameUIConstants::Margin,
                m_logoSize.height + GameUIConstants::Margin
                )
            );
        d2dContext->DrawTextLayout(
            Point2F(m_logoSize.width + 2.0f * GameUIConstants::Margin,
GameUIConstants::Margin),
            m_titleHeaderLayout.get(),
            m_textBrush.get()
            );
```

```
        d2dContext->DrawTextLayout(
            Point2F(GameUIConstants::Margin, m_titleBodyVerticalOffset),
            m_titleBodyLayout.get(),
            m_textBrush.get()
            );
    }

    // Draw text for number of hits, total shots, and time remaining
    if (game != nullptr)
    {
        // This section is only used after the game state has been
initialized.
        static const int bufferLength = 256;
        static wchar_t wsbuffer[bufferLength];
        int length = swprintf_s(
            wsbuffer,
            bufferLength,
            L"Hits:\t%10d\nShots:\t%10d\nTime:\t%8.1f",
            game->TotalHits(),
            game->TotalShots(),
            game->TimeRemaining()
            );

        // Draw the upper right portion of the HUD displaying total hits,
shots, and time remaining
        d2dContext->DrawText(
            wsbuffer,
            length,
            m_textFormatBody.get(),
            D2D1::RectF(
                windowBounds.Width - GameUIConstants::HudRightOffset,
                GameUIConstants::HudTopOffset,
                windowBounds.Width,
                GameUIConstants::HudTopOffset +
(GameUIConstants::HudBodyPointSize + GameUIConstants::Margin) * 3
                ),
            m_textBrush.get()
            );

        // Using the unicode characters starting at 0x2780 ( ① ) for the
consecutive levels of the game.
        // For completed levels start with 0x278A ( ❶ ) (This is 0x2780 +
10).
        uint32_t levelCharacter[6];
        for (uint32_t i = 0; i < 6; i++)
        {
            levelCharacter[i] = 0x2780 + i + ((static_cast<uint32_t>(game-
>LevelCompleted()) == i) ? 10 : 0);
        }
        length = swprintf_s(
            wsbuffer,
            bufferLength,
            L"%lc %lc %lc %lc %lc %lc",
            levelCharacter[0],
            levelCharacter[1],
```

```
            levelCharacter[2],
            levelCharacter[3],
            levelCharacter[4],
            levelCharacter[5]
            );
        // Create a new rectangle and draw the current level info text
inside
        d2dContext->DrawText(
            wsbuffer,
            length,
            m_textFormatBodySymbol.get(),
            D2D1::RectF(
                windowBounds.Width - GameUIConstants::HudRightOffset,
                GameUIConstants::HudTopOffset +
(GameUIConstants::HudBodyPointSize + GameUIConstants::Margin) * 3 +
GameUIConstants::Margin,
                windowBounds.Width,
                GameUIConstants::HudTopOffset +
(GameUIConstants::HudBodyPointSize + GameUIConstants::Margin) * 4
                ),
            m_textBrush.get()
            );

        if (game->IsActivePlay())
        {
            // Draw the move and fire rectangles
            ...
            // Draw the crosshairs
            ...
        }
    }
}
```

Breaking the method down further, this piece of the GameHud::Render method draws our move and fire rectangles with ID2D1RenderTarget::DrawRectangle, and crosshairs using two calls to ID2D1RenderTarget::DrawLine.

C++/WinRT

```
// Check if game is playing
if (game->IsActivePlay())
{
    // Draw a rectangle for the touch input for the move control.
    d2dContext->DrawRectangle(
        D2D1::RectF(
            0.0f,
            windowBounds.Height - GameUIConstants::TouchRectangleSize,
            GameUIConstants::TouchRectangleSize,
            windowBounds.Height
            ),
        m_textBrush.get()
        );
    // Draw a rectangle for the touch input for the fire control.
```

```
    d2dContext->DrawRectangle(
        D2D1::RectF(
            windowBounds.Width - GameUIConstants::TouchRectangleSize,
            windowBounds.Height - GameUIConstants::TouchRectangleSize,
            windowBounds.Width,
            windowBounds.Height
            ),
        m_textBrush.get()
        );

    // Draw the cross hairs
    d2dContext->DrawLine(
        D2D1::Point2F(windowBounds.Width / 2.0f -
GameUIConstants::CrossHairHalfSize,
            windowBounds.Height / 2.0f),
        D2D1::Point2F(windowBounds.Width / 2.0f +
GameUIConstants::CrossHairHalfSize,
            windowBounds.Height / 2.0f),
        m_textBrush.get(),
        3.0f
        );
    d2dContext->DrawLine(
        D2D1::Point2F(windowBounds.Width / 2.0f, windowBounds.Height / 2.0f
-
            GameUIConstants::CrossHairHalfSize),
        D2D1::Point2F(windowBounds.Width / 2.0f, windowBounds.Height / 2.0f
+
            GameUIConstants::CrossHairHalfSize),
        m_textBrush.get(),
        3.0f
        );
}
```

In the **GameHud::Render** method we store the logical size of the game window in the `windowBounds` variable. This uses the GetLogicalSize method of the **DeviceResources** class.

C++/WinRT

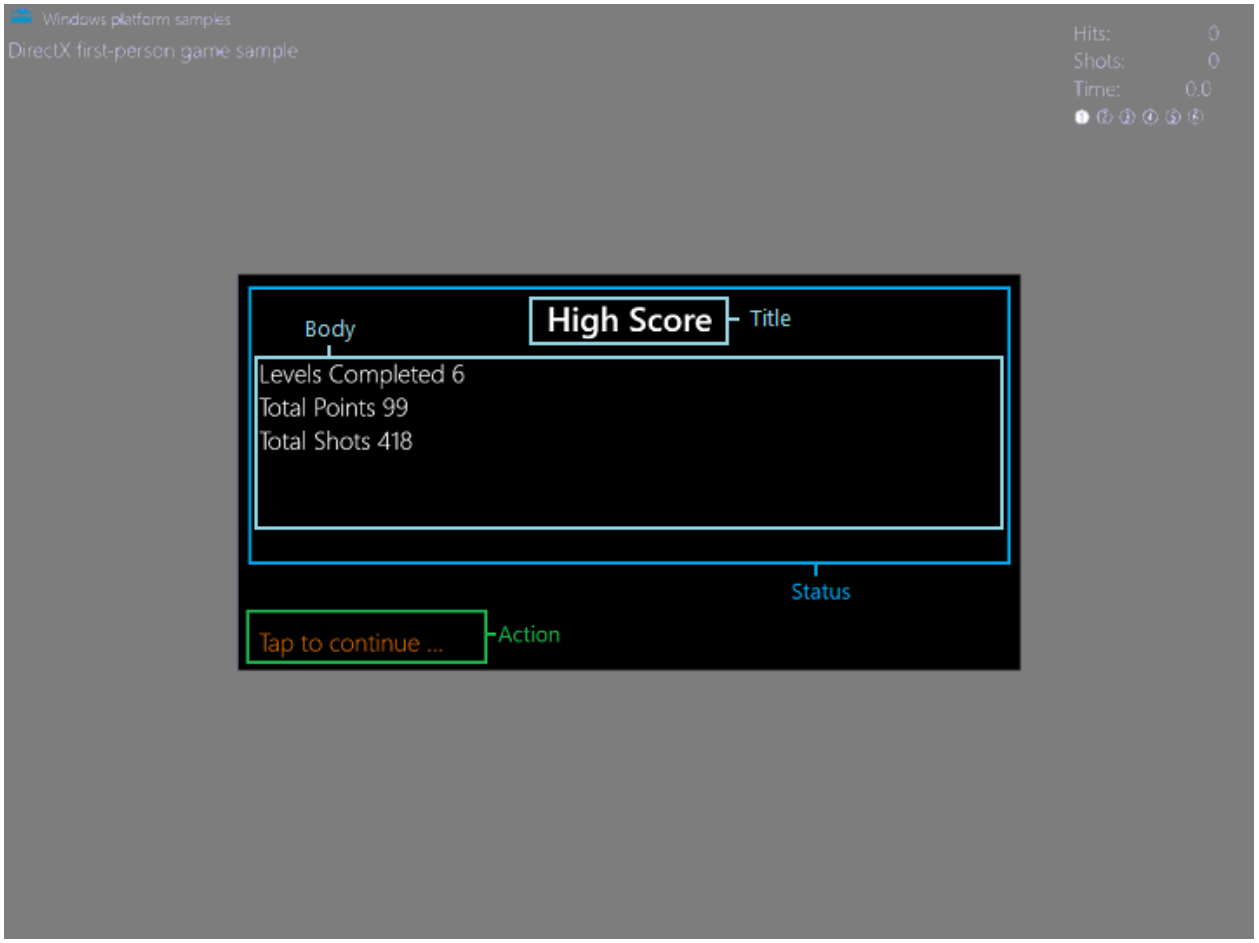
```
auto windowBounds = m_deviceResources->GetLogicalSize();
```

Obtaining the size of the game window is essential for UI programming. The size of the window is given in a measurement called DIPs (device independent pixels), where a DIP is defined as 1/96 of an inch. Direct2D scales the drawing units to actual pixels when the drawing occurs, doing so by using the Windows dots per inch (DPI) setting. Similarly, when you draw text using **DirectWrite**, you specify DIPs rather than points for the size of the font. DIPs are expressed as floating point numbers.

Displaying game state info

Besides the heads-up display, the sample game has an overlay that represents six game states. All states feature a large black rectangle primitive with text for the player to read. The move-look controller rectangles and crosshairs are not drawn because they are not active in these states.

The overlay is created using the GameInfoOverlay class, allowing us to switch out what text is displayed to align with the state of the game.



The overlay is broken up into two sections: **Status** and **Action**. The **Status** section is further broken down into **Title** and **Body** rectangles. The **Action** section only has one rectangle. Each rectangle has a different purpose.

- `titleRectangle` contains the title text.
- `bodyRectangle` contains the body text.
- `actionRectangle` contains the text that informs the player to take a specific action.

The game has six states that can be set. The state of the game conveyed using the **Status** portion of the overlay. The **Status** rectangles are updated using a number of methods corresponding with the following states.

- Loading

- Initial start/high score stats
- Level start
- Game paused
- Game over
- Game won

The **Action** portion of the overlay is updated using the GameInfoOverlay::SetAction method, allowing the action text to be set to one of the following.

- "Tap to play again..."
- "Level loading, please wait ..."
- "Tap to continue ..."
- None

> **Note**
>
> Both of these methods will be discussed further in the Representing game state section.

Depending on the what's going on in the game, the **Status** and **Action** section text fields are adjusted. Let's look at how we initialize and draw the overlay for these six states.

Initializing and drawing the overlay

The six **Status** states have a few things in common, making the resources and methods they need very similar. - They all use a black rectangle in the center of the screen as their background. - The displayed text is either **Title** or **Body** text. - The text uses the Segoe UI font and is drawn on top of the back rectangle.

The sample game has four methods that come into play when creating the overlay.

GameInfoOverlay::GameInfoOverlay

The GameInfoOverlay::GameInfoOverlay constructor initializes the overlay, maintaining the bitmap surface that we will use to display info to the player on. The constructor obtains a factory from the ID2D1Device object passed to it, which it uses to create an ID2D1DeviceContext that the overlay object itself can draw to. IDWriteFactory::CreateTextFormat

GameInfoOverlay::CreateDeviceDependentResources

GameInfoOverlay::CreateDeviceDependentResources is our method for creating brushes that will be used to draw our text. To do this, we obtain an ID2D1DeviceContext2 object which enables the creation and drawing of geometry, plus functionality such as ink and gradient mesh rendering. We then create a series of colored brushes using ID2D1SolidColorBrush to draw the following UI elements.

- Black brush for rectangle backgrounds
- White brush for status text
- Orange brush for action text

DeviceResources::SetDpi

The DeviceResources::SetDpi method sets the dots per inch of the window. This method gets called when the DPI is changed and needs to be readjusted which happens when the game window is resized. After updating the DPI, this method also callsDeviceResources::CreateWindowSizeDependentResources to make sure necessary resources are recreated every time the window is resized.

GameInfoOverlay::CreateWindowsSizeDependentResources

The GameInfoOverlay::CreateWindowsSizeDependentResources method is where all our drawing takes place. The following is an outline of the method's steps.

- Three rectangles are created to section off the UI text for the **Title**, **Body**, and **Action** text.

```
C++/WinRT
```

```
m_titleRectangle = D2D1::RectF(
    GameInfoOverlayConstant::SideMargin,
    GameInfoOverlayConstant::TopMargin,
    overlaySize.width - GameInfoOverlayConstant::SideMargin,
    GameInfoOverlayConstant::TopMargin +
GameInfoOverlayConstant::TitleHeight
    );
m_actionRectangle = D2D1::RectF(
    GameInfoOverlayConstant::SideMargin,
    overlaySize.height - (GameInfoOverlayConstant::ActionHeight +
GameInfoOverlayConstant::BottomMargin),
    overlaySize.width - GameInfoOverlayConstant::SideMargin,
    overlaySize.height - GameInfoOverlayConstant::BottomMargin
    );
m_bodyRectangle = D2D1::RectF(
    GameInfoOverlayConstant::SideMargin,
    m_titleRectangle.bottom + GameInfoOverlayConstant::Separator,
    overlaySize.width - GameInfoOverlayConstant::SideMargin,
```

```
    m_actionRectangle.top - GameInfoOverlayConstant::Separator
    );
```

- A Bitmap is created named `m_levelBitmap`, taking the current DPI into account using **CreateBitmap**.
- `m_levelBitmap` is set as our 2D render target using ID2D1DeviceContext::SetTarget.
- The Bitmap is cleared with every pixel made black using ID2D1RenderTarget::Clear.
- ID2D1RenderTarget::BeginDraw is called to initiate drawing.
- **DrawText** is called to draw the text stored in `m_titleString`, `m_bodyString`, and `m_actionString` in the appropriate rectangle using the corresponding **ID2D1SolidColorBrush**.
- ID2D1RenderTarget::EndDraw is called to stop all drawing operations on `m_levelBitmap`.
- Another Bitmap is created using **CreateBitmap** named `m_tooSmallBitmap` to use as a fallback, showing only if the display configuration is too small for the game.
- Repeat process for drawing on `m_levelBitmap` for `m_tooSmallBitmap`, this time only drawing the string `Paused` in the body.

Now all we need are six methods to fill the text of our six overlay states!

Representing game state

Each of the six overlay states in the game has a corresponding method in the **GameInfoOverlay** object. These methods draw a variation of the overlay to communicate explicit info to the player about the game itself. This communication is represented with a **Title** and **Body** string. Because the sample already configured the resources and layout for this info when it was initialized and with the GameInfoOverlay::CreateDeviceDependentResources method, it only needs to provide the overlay state-specific strings.

The **Status** portion of the overlay is set with a call to one of the following methods.

Expand table

Game state	Status set method	Status fields
Loading	GameInfoOverlay::SetGameLoading	**Title** Loading Resources **Body** Incrementally prints "." to imply loading activity.
Initial start/high score stats	GameInfoOverlay::SetGameStats	**Title** High Score **Body** Levels Completed # Total Points # Total Shots #
Level start	GameInfoOverlay::SetLevelStart	**Title** Level # **Body** Level objective description.
Game paused	GameInfoOverlay::SetPause	**Title** Game Paused **Body** None
Game over	GameInfoOverlay::SetGameOver	**Title** Game Over **Body** Levels Completed # Total Points # Total Shots # Levels Completed # High Score #
Game won	GameInfoOverlay::SetGameOver	**Title** You WON! **Body** Levels Completed # Total Points # Total Shots # Levels Completed # High Score #

With the **GameInfoOverlay::CreateWindowSizeDependentResources** method, the sample declared three rectangular areas that correspond to specific regions of the overlay.

With these areas in mind, let's look at one of the state-specific methods, **GameInfoOverlay::SetGameStats**, and see how the overlay is drawn.

C++/WinRT

```
void GameInfoOverlay::SetGameStats(int maxLevel, int hitCount, int
shotCount)
{
    int length;

    auto d2dContext = m_deviceResources->GetD2DDeviceContext();

    d2dContext->SetTarget(m_levelBitmap.get());
    d2dContext->BeginDraw();
    d2dContext->SetTransform(D2D1::Matrix3x2F::Identity());
    d2dContext->FillRectangle(&m_titleRectangle, m_backgroundBrush.get());
    d2dContext->FillRectangle(&m_bodyRectangle, m_backgroundBrush.get());
    m_titleString = L"High Score";

    d2dContext->DrawText(
        m_titleString.c_str(),
        m_titleString.size(),
        m_textFormatTitle.get(),
        m_titleRectangle,
        m_textBrush.get()
        );
    length = swprintf_s(
        wsbuffer,
        bufferLength,
        L"Levels Completed %d\nTotal Points %d\nTotal Shots %d",
        maxLevel,
        hitCount,
        shotCount
        );
    m_bodyString = std::wstring(wsbuffer, length);
    d2dContext->DrawText(
        m_bodyString.c_str(),
        m_bodyString.size(),
        m_textFormatBody.get(),
        m_bodyRectangle,
        m_textBrush.get()
        );

    // We ignore D2DERR_RECREATE_TARGET here. This error indicates that the
device
    // is lost. It will be handled during the next call to Present.
    HRESULT hr = d2dContext->EndDraw();
    if (hr != D2DERR_RECREATE_TARGET)
    {
        // The D2DERR_RECREATE_TARGET indicates there has been a problem
with the underlying
        // D3D device. All subsequent rendering will be ignored until the
device is recreated.
        // This error will be propagated and the appropriate D3D error will
be returned from the
        // swapchain->Present(...) call. At that point, the sample will
recreate the device
```

```
        // and all associated resources. As a result, the
D2DERR_RECREATE_TARGET doesn't
        // need to be handled here.
        winrt::check_hresult(hr);
    }
}
```

Using the Direct2D device context that the **GameInfoOverlay** object initialized, this method fills the title and body rectangles with black using the background brush. It draws the text for the "High Score" string to the title rectangle and a string containing the updates game state information to the body rectangle using the white text brush.

The action rectangle is updated by a subsequent call to GameInfoOverlay::SetAction from a method on the **GameMain** object, which provides the game state info needed by **GameInfoOverlay::SetAction** to determine the right message to the player, such as "Tap to continue".

The overlay for any given state is chosen in the GameMain::SetGameInfoOverlay method like this:

C++/WinRT

```
void GameMain::SetGameInfoOverlay(GameInfoOverlayState state)
{
    m_gameInfoOverlayState = state;
    switch (state)
    {
    case GameInfoOverlayState::Loading:
        m_uiControl->SetGameLoading(m_loadingCount);
        break;

    case GameInfoOverlayState::GameStats:
        m_uiControl->SetGameStats(
            m_game->HighScore().levelCompleted + 1,
            m_game->HighScore().totalHits,
            m_game->HighScore().totalShots
            );
        break;

    case GameInfoOverlayState::LevelStart:
        m_uiControl->SetLevelStart(
            m_game->LevelCompleted() + 1,
            m_game->CurrentLevel()->Objective(),
            m_game->CurrentLevel()->TimeLimit(),
            m_game->BonusTime()
            );
        break;

    case GameInfoOverlayState::GameOverCompleted:
        m_uiControl->SetGameOver(
```

```
            true,
            m_game->LevelCompleted() + 1,
            m_game->TotalHits(),
            m_game->TotalShots(),
            m_game->HighScore().totalHits
            );
        break;

    case GameInfoOverlayState::GameOverExpired:
        m_uiControl->SetGameOver(
            false,
            m_game->LevelCompleted(),
            m_game->TotalHits(),
            m_game->TotalShots(),
            m_game->HighScore().totalHits
            );
        break;

    case GameInfoOverlayState::Pause:
        m_uiControl->SetPause(
            m_game->LevelCompleted() + 1,
            m_game->TotalHits(),
            m_game->TotalShots(),
            m_game->TimeRemaining()
            );
        break;
    }
}
```

Now the game has a way to communicate text info to the player based on game state, and we have a way of switching what's displayed to them throughout the game.

Next steps

In the next topic, Adding controls, we look at how the player interacts with the sample game, and how input changes game state.

Feedback

Add controls

Article • 10/20/2022

> **Note**
>
> This topic is part of the Create a simple Universal Windows Platform (UWP) game with DirectX tutorial series. The topic at that link sets the context for the series.

[Updated for UWP apps on Windows 10. For Windows 8.x articles, see the archive]

A good Universal Windows Platform (UWP) game supports a wide variety of interfaces. A potential player might have Windows 10 on a tablet with no physical buttons, a PC with a game controller attached, or the latest desktop gaming rig with a high-performance mouse and gaming keyboard. In our game the controls are implemented in the MoveLookController class. This class aggregates all three types of input (mouse and keyboard, touch, and gamepad) into a single controller. The end result is a first-person shooter that uses genre standard move-look controls that work with multiple devices.

> **Note**
>
> For more info about controls, see Move-look controls for games and Touch controls for games.

Objective

At this point we have a game that renders, but we can't move our player around or shoot the targets. We'll take a look at how our game implements first person shooter move-look controls for the following types of input in our UWP DirectX game.

- Mouse and keyboard
- Touch
- Gamepad

> **Note**
>
> If you haven't downloaded the latest game code for this sample, go to Direct3D sample game. This sample is part of a large collection of UWP feature samples.